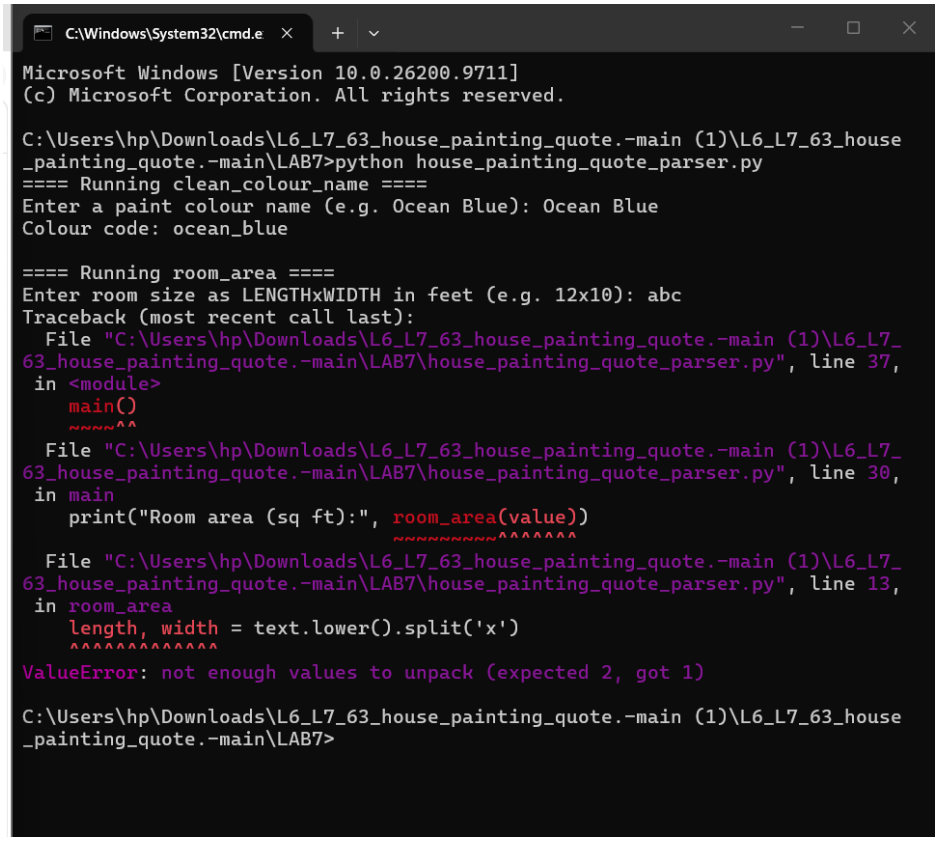
Lab 7: Fuzzing for Input Validation Bugs

Name: Shraddha | SRN: PES1UG25AM812
Module: house_painting_quote_parser.py | Tests: test_house_painting_quote_parser.py (Hypothesis + pytest)

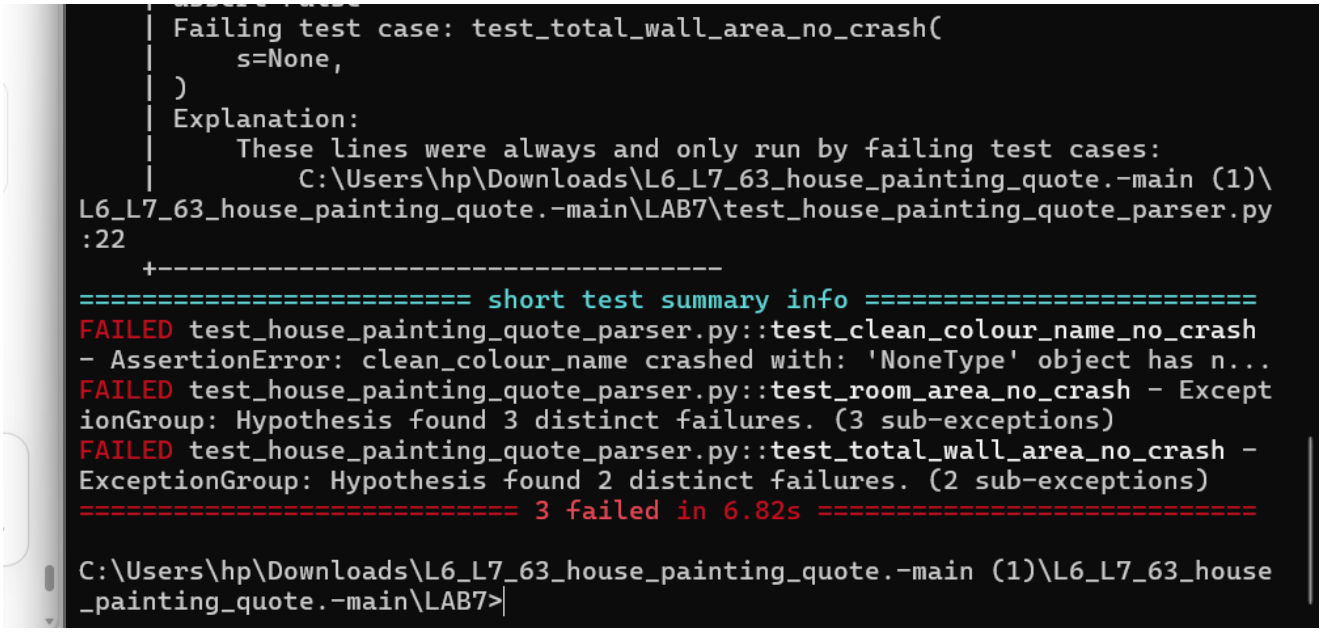
SS1: Buggy module run

python house_painting_quote_parser.py with bad input 'abc' for room size. The program crashes with ValueError: not enough values to unpack.



SS2: Hypothesis tests failing on the buggy module

pytest test_house_painting_quote_parser.py: 3 failed. Hypothesis found inputs such as None that crash all three functions.



SS3: Fixed module run

python house_painting_quote_parser.py with the same bad input. No crash: room area gives 0 and 'abc' is skipped in the wall total (120;abc;95 gives 215).

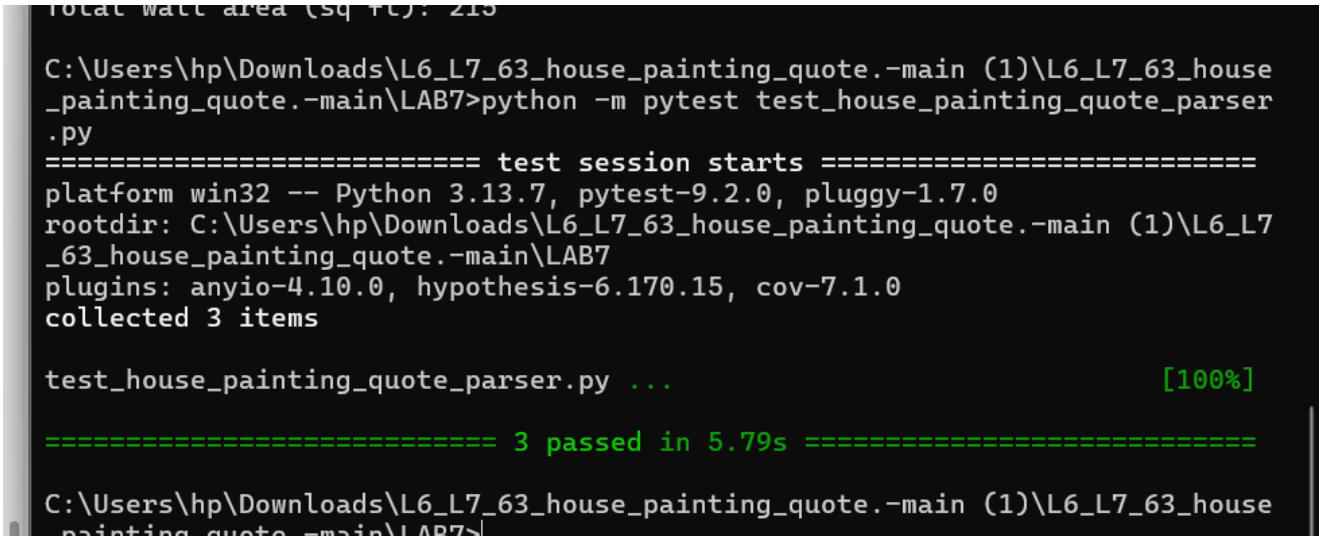
```
C:\Users\hp\...\LAB7>python house_painting_quote_parser.py
==== Running clean_colour_name ====
Enter a paint colour name (e.g. Ocean Blue): Ocean Blue
Colour code: ocean_blue

==== Running room_area ====
Enter room size as LENGTHxWIDTH in feet (e.g. 12x10): abc
Room area (sq ft): 0

==== Running total_wall_area ====
Enter wall areas separated by ; (e.g. 120;95;110): 120;abc;95
Total wall area (sq ft): 215
```

SS4: Hypothesis tests passing on the fixed module

pytest test_house_painting_quote_parser.py: 3 passed.



Bugs found and how they were fixed

Function	Bad input found	Original problem	Fix
clean_colour_name	None, numbers, tabs	AttributeError: None has no strip(); tabs not replaced	Return '' if not a string; use '_'.join(name.lower().split())
room_area	'', 'abc', '12', '12x10x5', '-5x3'	ValueError when unpacking; negative areas accepted	Check exactly one 'x'; accept only digits 0-9; return 0 if invalid
total_wall_area	'', '120;;95', '120;abc', None	ValueError from int(); None crashes	Skip empty or invalid parts; return the sum of valid ones (0 if none)

Reflection

1. What bugs did fuzzing find?

Hypothesis found that all three functions trusted their input too much. clean_colour_name crashed on None (AttributeError). room_area crashed on text without exactly one 'x', such as '', 'abc' or '12x10x5' (ValueError), and it accepted negative sizes. total_wall_area crashed on empty parts, extra semicolons or non-numbers (ValueError) and on None. I would not have thought of many of these inputs myself.

2. How did you fix them?

I added input checks to each function. If the input is not a string, the function returns a safe value (empty text or 0). room_area now checks there is exactly one 'x' and that both parts are whole numbers (digits 0-9 only). total_wall_area skips empty or invalid parts and adds only the valid numbers. clean_colour_name uses split() so tabs and extra spaces are also handled. After the fix, all 3 Hypothesis tests pass.

3. Why is fuzzing useful compared to normal unit tests?

A normal unit test only checks the cases I think of. Fuzzing automatically tries hundreds of random and unexpected inputs, so it finds edge cases I did not imagine, and Hypothesis shrinks each failure to the smallest failing input (for example s=None), which makes the bug easy to understand and fix. It makes the code more robust.